Dead Letter Queue - Documented Code

```python
import json  # For serializing dead letters to JSON format
import logging  # Professional logging instead of print statements
from dataclasses import dataclass, field  # Auto-generate class methods
from datetime import datetime  # Timestamp each failure
from pathlib import Path  # Modern file path handling
from typing import Dict, Any, Iterable, Tuple  # Type hints for better code clarity
from collections import Counter  # Count occurrences of error types
from enum import Enum  # Create fixed constants for pipeline stages

# Configure logging to show INFO level messages
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


# ============================================================
# STAGE ENUM - Pipeline stage constants
# ============================================================

class Stage(str, Enum):
    """Fixed constants for pipeline stages - prevents typos"""
    VALIDATE = "validate"  # Validation stage
    TRANSFORM = "transform"  # Transformation stage
    LOAD = "load"  # Loading stage


# ============================================================
# ERROR HIERARCHY - Custom exceptions for pipeline
# ============================================================

class PipelineError(Exception):
    """Base exception for all pipeline errors"""
    pass


class ValidationError(PipelineError):
    """Raised when data validation fails"""
    pass


class TransformError(PipelineError):
    """Raised when data transformation fails"""
    pass
```

```python
# ============================================================
# DEAD LETTER SCHEMA - Structure of a failed record
# ============================================================

@dataclass  # Auto-generates __init__, __repr__, __eq__ methods
class DeadLetter:
    """Complete information about a failed record"""
    timestamp: datetime  # When the failure occurred
    record_id: str  # Unique identifier for the record
    line_number: int  # Line number in source file (for debugging)
    stage: Stage  # Which pipeline stage failed
    raw_record: str  # Original record as JSON string (for replay)
    error_type: str  # Exception class name (e.g., "ValidationError")
    error_message: str  # Human-readable error description
    context: Dict[str, Any]  # Additional metadata about the failure
    retry_count: int = 0  # Number of retry attempts (default: 0)


# ============================================================
# PIPELINE STATISTICS - Track success/failure metrics
# ============================================================

@dataclass
class PipelineStats:
    """Tracks pipeline processing metrics"""
    total_processed: int = 0  # Total records processed
    successful: int = 0  # Successfully processed records
    failed: int = 0  # Failed records (sent to DLQ)
    error_types: Counter = field(default_factory=Counter)  # Count by error type

    def record_success(self):
        """Increment counters when a record succeeds"""
        self.total_processed += 1  # Increment total
        self.successful += 1  # Increment success count

    def record_failure(self, error_type: str):
        """Track a failure and categorize by error type"""
        self.total_processed += 1  # Increment total
        self.failed += 1  # Increment failure count
        self.error_types[error_type] += 1  # Count this error type

    def summary(self) -> str:
        """Generate human-readable statistics summary"""
        # Calculate success rate (prevent division by zero)
        success_rate = (self.successful / self.total_processed * 100) \
                       if self.total_processed > 0 else 0
```

```
        # Return formatted statistics string
        return f"""
Pipeline Statistics:
  Total Processed: {self.total_processed}
  Successful: {self.successful} ({success_rate:.1f}%)
  Failed: {self.failed}
  Error Breakdown: {dict(self.error_types)}
        """.strip()


# ============================================================
# DEAD LETTER QUEUE - Core DLQ implementation
# ============================================================

class DeadLetterQueue:
    """
    Append-only, crash-safe storage for failed records.
    CRITICAL: This class must NEVER crash the main pipeline.
    """

    def __init__(self, path: str, alert_threshold: int = 10):
        """
        Initialize the DLQ

        Args:
            path: File path for DLQ storage (JSONL format)
            alert_threshold: Send alert after this many failures
        """
        self.path = Path(path)  # Convert string to Path object
        # Create all parent directories if they don't exist
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.alert_threshold = alert_threshold  # Store threshold
        self.failure_count = 0  # Initialize failure counter

    def write(self, dead_letter: DeadLetter) -> None:
        """
        Write a failed record to DLQ.
        NEVER raises exceptions - all failures are logged only.

        Args:
            dead_letter: DeadLetter object to persist
        """
        try:
            # Open file in append mode ("a" = append, don't erase existing)
            with self.path.open("a", encoding="utf-8") as f:
                # Convert DeadLetter to dict, then to JSON
```

```
                # default=str handles datetime serialization
                json.dump(dead_letter.__dict__, f, default=str)
                f.write("\n")  # Add newline (creates JSONL format)

            self.failure_count += 1  # Increment failure counter

            # Send alert if threshold exceeded
            if self.failure_count >= self.alert_threshold:
                self._send_alert()  # Call alert method

        except Exception as e:
            # CRITICAL: Never propagate DLQ write failures
            # Just log the error and continue
            logger.error(f"[DLQ ERROR] Failed to write dead letter: {e}")

    def _send_alert(self):
        """Send alert when failure threshold is reached"""
        # Log warning (in production: send to PagerDuty, Slack, etc.)
        logger.warning(
            f"[ALERT] DLQ threshold reached: {self.failure_count} failures"
        )

    def read_all(self) -> list[DeadLetter]:
        """
        Read all dead letters from DLQ for analysis/replay

        Returns:
            List of DeadLetter objects
        """
        dead_letters = []  # Initialize empty list

        if not self.path.exists():  # If DLQ file doesn't exist
            return dead_letters  # Return empty list

        # Open file in read mode
        with self.path.open("r", encoding="utf-8") as f:
            for line in f:  # Read file line by line (JSONL format)
                try:
                    data = json.loads(line)  # Parse JSON string to dict

                    # Convert timestamp string back to datetime object
                    if isinstance(data['timestamp'], str):
                        data['timestamp'] = datetime.fromisoformat(
                            data['timestamp']
                        )
```

```
                    # Convert stage string back to Stage enum
                    if isinstance(data['stage'], str):
                        data['stage'] = Stage(data['stage'])

                    # Create DeadLetter object and add to list
                    dead_letters.append(DeadLetter(**data))

                except Exception as e:
                    # If parsing fails, log and skip this line
                    logger.error(f"Failed to parse dead letter: {e}")

        return dead_letters  # Return all successfully parsed dead letters

    def clear(self) -> None:
        """Clear DLQ file (use after successful replay)"""
        if self.path.exists():  # If file exists
            self.path.unlink()  # Delete the file
            logger.info("DLQ cleared")  # Log the action


# ============================================================
# VALIDATION LOGIC - Validate and normalize records
# ============================================================

def validate_record(record: Dict[str, Any]) -> Dict[str, Any]:
    """
    Validate and normalize a record.

    Args:
        record: Dictionary containing record data

    Returns:
        Validated and normalized record

    Raises:
        ValidationError: If validation fails
    """
    # Check if "name" field exists and is not empty
    if "name" not in record or not record["name"]:
        raise ValidationError("name is required")

    # Get age field (returns None if doesn't exist)
    age = record.get("age")

    # If age is None or empty string, set to None (acceptable)
    if age is None or age == "":
```

```
        record["age"] = None
    else:
        # Convert to string and check if all characters are digits
        if not str(age).isdigit():
            # If not digits, raise validation error
            raise ValidationError(f"age must be an integer, got: {age}")
        # Convert to integer
        record["age"] = int(age)

    return record  # Return validated record


# ============================================================
# RECORD PROCESSOR - Main pipeline loop
# ============================================================

def process_records(
    records: Iterable[Tuple[int, Dict[str, Any]]],
    dlq: DeadLetterQueue,
    stats: PipelineStats
) -> None:
    """
    Process records one by one.
    Failures go to DLQ, processing never stops.

    Args:
        records: Iterable of (line_number, record_dict) tuples
        dlq: DeadLetterQueue instance for storing failures
        stats: PipelineStats instance for tracking metrics
    """
    # Iterate through each record
    for line_number, record in records:
        # Get record ID (fallback to line number if no "id" field)
        record_id = record.get("id", f"line_{line_number}")

        try:
            # ---- VALIDATE STAGE ----
            validated_record = validate_record(record)  # Validate record

            # ---- TRANSFORM STAGE ----
            # (Your transformation logic would go here)

            # ---- LOAD STAGE ----
            # (Your load logic would go here)

            # Log success
```

```
            logger.info(f"[SUCCESS] {record_id}: {validated_record}")
            stats.record_success()  # Update success counter

        except PipelineError as e:  # Catch only pipeline errors
            # ---- HANDLE FAILURE ----

            # Create dead letter with all failure context
            dead_letter = DeadLetter(
                timestamp=datetime.utcnow(),  # Current UTC time
                record_id=record_id,  # Record identifier
                line_number=line_number,  # Line number in source
                stage=Stage.VALIDATE,  # Which stage failed
                raw_record=json.dumps(record),  # Original record as JSON
                error_type=type(e).__name__,  # Exception class name
                error_message=str(e),  # Human-readable error
                context={"original_record": record}  # Additional metadata
            )

            dlq.write(dead_letter)  # Write to DLQ (never crashes)
            logger.error(f"[FAILED] {record_id}: {e}")  # Log failure
            stats.record_failure(type(e).__name__)  # Update stats

            # CRITICAL: Continue to next record (don't stop pipeline)
            continue


# ============================================================
# DLQ REPLAY - Reprocess failed records
# ============================================================

def replay_dlq(dlq: DeadLetterQueue, stats: PipelineStats) -> None:
    """
    Attempt to reprocess all records from DLQ.
    Use after fixing bugs or updating validation rules.

    Args:
        dlq: DeadLetterQueue to replay from
        stats: PipelineStats for tracking replay results
    """
    dead_letters = dlq.read_all()  # Read all failed records
    logger.info(f"Replaying {len(dead_letters)} records from DLQ")

    # Convert dead letters back to (line_number, record) format
    records_to_replay = [
        (dl.line_number, json.loads(dl.raw_record))  # Parse JSON back to dict
        for dl in dead_letters
```

```
    ]

    # Create separate DLQ for records that still fail
    replay_dlq_path = dlq.path.parent / "failed_replay.jsonl"
    replay_dlq_queue = DeadLetterQueue(str(replay_dlq_path))

    # Reprocess all records
    process_records(records_to_replay, replay_dlq_queue, stats)

    # Log completion with path to still-failing records
    logger.info(
        f"Replay complete. Check {replay_dlq_path} for still-failing records"
    )


# ============================================================
# MAIN FUNCTION - Entry point
# ============================================================

def main():
    """Main function demonstrating DLQ usage"""

    # Sample records with various validation scenarios
    records = [
        (1, {"id": "rec_001", "name": "Alice", "age": "25"}),      #  Valid
        (2, {"id": "rec_002", "name": "Bob", "age": "invalid"}),   #  Invalid age
        (3, {"id": "rec_003", "name": "Charlie", "age": ""}),      #  Empty age OK
        (4, {"id": "rec_004", "name": "Dave", "age": None}),       #  None age OK
        (5, {"id": "rec_005", "name": "Eve", "age": "42"}),        #  Valid
        (6, {"id": "rec_006", "name": "", "age": "30"}),           #  Missing name
    ]

    # Initialize DLQ (will alert after 3 failures)
    dlq = DeadLetterQueue("dead_letter/failed.jsonl", alert_threshold=3)

    # Initialize statistics tracker
    stats = PipelineStats()

    # Process all records
    logger.info("Starting pipeline...")
    process_records(records, dlq, stats)

    # Print statistics summary
    logger.info("\n" + stats.summary())

    # Optionally replay DLQ (uncomment to test)
```

```
    # replay_dlq(dlq, stats)


# Run main function if script is executed directly
if __name__ == "__main__":
    main()
```

Expected Output

```
INFO:__main__:Starting pipeline...
INFO:__main__:[SUCCESS] rec_001: {'id': 'rec_001', 'name': 'Alice', 'age': 25}
ERROR:__main__:[FAILED] rec_002: age must be an integer, got: invalid
INFO:__main__:[SUCCESS] rec_003: {'id': 'rec_003', 'name': 'Charlie', 'age': None}
INFO:__main__:[SUCCESS] rec_004: {'id': 'rec_004', 'name': 'Dave', 'age': None}
WARNING:__main__:[ALERT] DLQ threshold reached: 3 failures
ERROR:__main__:[FAILED] rec_006: name is required
INFO:__main__:
Pipeline Statistics:
  Total Processed: 6
  Successful: 4 (66.7%)
  Failed: 2
  Error Breakdown: {'ValidationError': 2}
```

DLQ File Output (dead_letter/failed.jsonl)


```
{"timestamp": "2025-02-07T10:30:45.123456", "record_id": "rec_002", "line_number": 2, "stage
{"timestamp": "2025-02-07T10:30:45.234567", "record_id": "rec_006", "line_number": 6, "stage
```


Quick Usage Guide

1. Initialize DLQ

```
dlq = DeadLetterQueue("path/to/failed.jsonl", alert_threshold=10)
```

2. Process Records

```
stats = PipelineStats()
process_records(your_records, dlq, stats)
print(stats.summary())
```

3. Read DLQ

```
failures = dlq.read_all()
for f in failures:
    print(f"{f.record_id}: {f.error_message}")
```

4. Replay DLQ

```
replay_dlq(dlq, stats)  # After fixing the bug
```

5. Clear DLQ

```
dlq.clear()  # After successful replay
```